COMPUTER NETWORKS

Subject Code: UE24CS252B

Dynamic Host Blocking System Using Software Defined Networking

Name: Dhruva Girish
SRN : PES2UG24AM055
Class : Sem-4 AIML A Sec
Date : 17/04/2026

TABLE OF CONTENTS:

- **Problem Statement**
- **SDN Architecture**
- **Mininet Topology Design**
- **Controller Implementation**
- **Performance Evaluation**
- **Python Script**
- **Demo Screenshots**
- **GitHub Repository**
- **Conclusion**

1. Problem Statement

Traditional networks rely on static firewall rules that cannot dynamically adapt to suspicious traffic behavior. In modern networks, malicious hosts may generate excessive traffic or abnormal patterns that must be detected and blocked automatically.

Software Defined Networking (SDN) provides centralized control over network behavior through a programmable controller. This allows dynamic monitoring of network traffic and the ability to update switch forwarding rules in real time.

The goal of this project is to implement a Dynamic Host Blocking System using SDN. The controller monitors network traffic and detects suspicious activity based on packet behavior. When a host exceeds a predefined threshold of packets, the controller installs a blocking rule in the OpenFlow switch, preventing further communication from that host.

This project demonstrates how SDN can be used to implement dynamic security policies such as traffic monitoring, host blocking, and flow rule management.

2. SDN Architecture

Software Defined Networking separates the network into two main components:

Control Plane

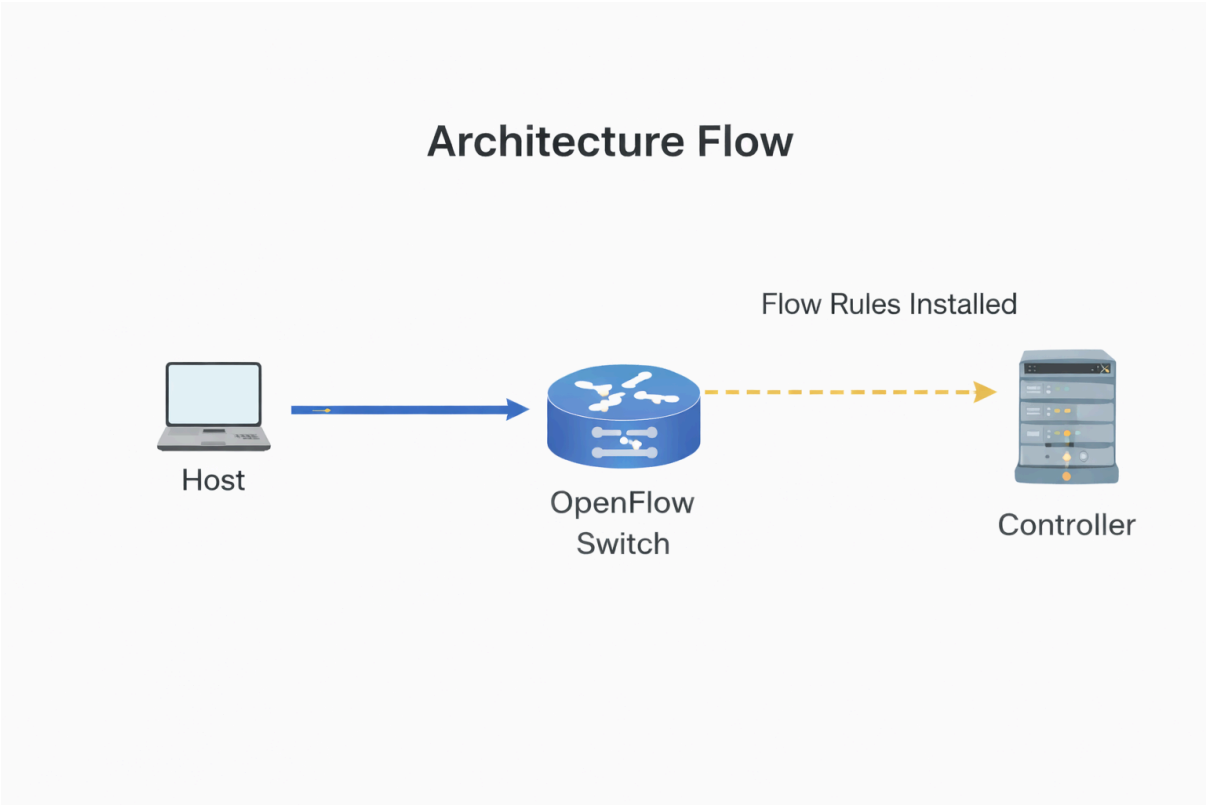
The control plane is responsible for making decisions about how packets should be forwarded. In this project, the control plane is implemented using the **POX controller**.

Data Plane

The data plane consists of network switches that forward packets according to rules installed by the controller. In this project, the data plane is implemented using **OpenFlow switches in Mininet**.

Communication Protocol

The controller communicates with the switch using the **OpenFlow protocol**, which allows the controller to install, modify, or remove flow rules dynamically.

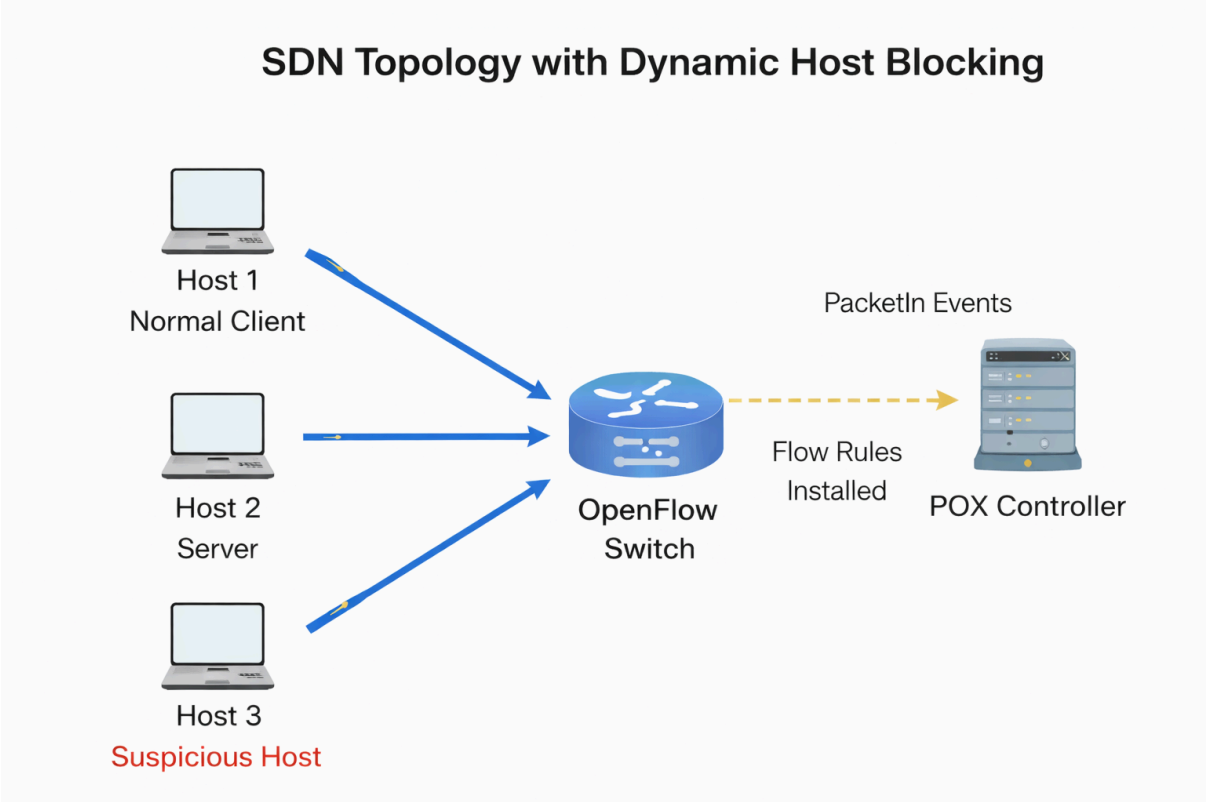


The controller analyzes packets received from the switch and decides whether to forward or block the traffic.

3. Mininet Topology Design

The network topology used in this project is a simple single-switch topology created using Mininet.

Topology:



Components:

Component	Description
h1	Normal client
h2	Server
h3	Suspicious host (attacker)
s1	OpenFlow switch
controller	POX controller implementing security logic

The topology allows the controller to monitor traffic between hosts and dynamically block malicious hosts.

4. Controller Implementation

The controller is implemented using Python in the POX SDN framework. The controller listens for **PacketIn events** generated by the OpenFlow switch whenever it encounters an unknown flow.

The controller performs the following steps:

1. Receives packet information from the switch
2. Extracts the source IP address
3. Maintains a packet count for each host
4. Detects suspicious behavior when packet count exceeds a threshold
5. Installs a flow rule to block the suspicious host

The threshold used in this implementation is **20 packets**.

Once the threshold is exceeded, the controller dynamically installs a drop rule in the switch to block further packets from that host.

5. Flow Rule Management

Flow rules are installed in the OpenFlow switch using the OpenFlow protocol.

A typical flow rule contains three components:

Component	Description
Match	Packet characteristics used to identify traffic
Action	Operation to perform on matched packets
Priority	Determines rule precedence

Flow rules can be verified using the following Mininet command:

```
dpctl dump-flows
```

6. Functionality Implementation

The system implements the following functionalities:

Traffic Monitoring

The controller continuously monitors packet flow between hosts.

Suspicious Activity Detection

Packet counts are tracked for each host. If the packet count exceeds the threshold, the host is considered suspicious.

Dynamic Host Blocking

Once suspicious behavior is detected, the controller installs a drop rule in the switch to block that host.

Logging

The controller logs all events, including packet counts and detection messages.

Example controller log output:

Traffic from 10.0.0.3 count=21

ALERT: Suspicious host detected

Blocking host 10.0.0.3

7. Performance Evaluation

Network performance was evaluated using latency and throughput measurements.

Latency Measurement

Latency was measured using the ping command:

```
h1 ping h2
```

This measures the round-trip time between hosts.

Throughput Measurement

Throughput was measured using the iperf tool.

Server:

```
h2 iperf -s
```

Client:

```
h1 iperf -c h2
```

This measures the bandwidth between hosts.

Flow Table Observation

Flow rules were examined using:

```
dpctl dump-flows
```

This confirmed that the blocking rule was successfully installed in the switch.

8. Python Script

```
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.packet import ethernet
from pox.lib.packet import ipv4
from collections import defaultdict

log = core.getLogger()

mac_table = {}
packet_count = defaultdict(int)
blocked_hosts = set()

THRESHOLD = 20


def _handle_PacketIn(event):

    packet = event.parsed
    if not packet:
        return

    src = packet.src
    dst = packet.dst

    mac_table[src] = event.port

    ip_packet = packet.find('ipv4')

    if ip_packet:
```

```
        src_ip = str(ip_packet.srcip)

        if src_ip == "10.0.0.3":

            packet_count[src_ip] += 1
            log.info("Traffic from %s count=%s", src_ip, packet_count[src_ip])

            if src_ip not in blocked_hosts and packet_count[src_ip] > THRESHOLD:

                log.warning("================================")
                log.warning("ALERT: Suspicious host detected")
                log.warning("Blocking host %s", src_ip)
                log.warning("================================")

                blocked_hosts.add(src_ip)

                msg = of.ofp_flow_mod()
                msg.match.dl_type = 0x0800
                msg.match.nw_src = src_ip
                msg.priority = 100
                msg.actions = []

                event.connection.send(msg)

                return

    if dst in mac_table:

        msg = of.ofp_packet_out()
        msg.data = event.ofp
        msg.actions.append(of.ofp_action_output(port=mac_table[dst]))
        msg.in_port = event.port

        event.connection.send(msg)

    else:

        msg = of.ofp_packet_out()
        msg.data = event.ofp
        msg.actions.append(of.ofp_action_output(port=of.OFPP_FLOOD))
        msg.in_port = event.port

        event.connection.send(msg)


def launch():
    log.info("Dynamic Host Blocking Controller Started")
    core.openflow.addListenerByName("PacketIn", _handle_PacketIn)
```

9. Demo Screenshots

Controller startup:

```
dhruva@dhruva:~/pox$ cd ~/pox
./pox.py openflow.of_01 dynamic_block
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:dynamic_block:Dynamic Host Blocking Controller Started
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.10.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
```

Mininet topology creation:

```
dhruva@dhruva:~/pox$ sudo mn --topo single,3 --controller=remote,ip=127.0.0.1,port=6633
[sudo] password for dhruva:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

Pingall connectivity test:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet>
```

Attack simulation (`h3 ping h1`):

```
mininet> h3 ping h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=2.00 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.57 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=2.00 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=1.35 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=1.91 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=1.88 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=1.50 ms
64 bytes from 10.0.0.1: icmp_seq=8 ttl=64 time=2.04 ms
64 bytes from 10.0.0.1: icmp_seq=9 ttl=64 time=1.48 ms
64 bytes from 10.0.0.1: icmp_seq=10 ttl=64 time=1.48 ms
64 bytes from 10.0.0.1: icmp_seq=11 ttl=64 time=2.97 ms
64 bytes from 10.0.0.1: icmp_seq=12 ttl=64 time=1.71 ms
64 bytes from 10.0.0.1: icmp_seq=13 ttl=64 time=1.96 ms
64 bytes from 10.0.0.1: icmp_seq=14 ttl=64 time=1.91 ms
64 bytes from 10.0.0.1: icmp_seq=15 ttl=64 time=1.77 ms
64 bytes from 10.0.0.1: icmp_seq=16 ttl=64 time=1.94 ms
```

Controller detection logs:

```
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:dynamic_block:Traffic from 10.0.0.3 count=1
INFO:dynamic_block:Traffic from 10.0.0.3 count=2
INFO:dynamic_block:Traffic from 10.0.0.3 count=3
INFO:dynamic_block:Traffic from 10.0.0.3 count=4
INFO:dynamic_block:Traffic from 10.0.0.3 count=5
INFO:dynamic_block:Traffic from 10.0.0.3 count=6
INFO:dynamic_block:Traffic from 10.0.0.3 count=7
INFO:dynamic_block:Traffic from 10.0.0.3 count=8
INFO:dynamic_block:Traffic from 10.0.0.3 count=9
INFO:dynamic_block:Traffic from 10.0.0.3 count=10
INFO:dynamic_block:Traffic from 10.0.0.3 count=11
INFO:dynamic_block:Traffic from 10.0.0.3 count=12
INFO:dynamic_block:Traffic from 10.0.0.3 count=13
INFO:dynamic_block:Traffic from 10.0.0.3 count=14
INFO:dynamic_block:Traffic from 10.0.0.3 count=15
INFO:dynamic_block:Traffic from 10.0.0.3 count=16
INFO:dynamic_block:Traffic from 10.0.0.3 count=17
INFO:dynamic_block:Traffic from 10.0.0.3 count=18
INFO:dynamic_block:Traffic from 10.0.0.3 count=19
INFO:dynamic_block:Traffic from 10.0.0.3 count=20
INFO:dynamic_block:Traffic from 10.0.0.3 count=21
WARNING:dynamic_block:==================================
WARNING:dynamic_block:ALERT: Suspicious host detected
WARNING:dynamic_block:Blocking host 10.0.0.3
WARNING:dynamic_block:==================================
```

Flow table rule (`dpctl dump-flows`):

```
mininet> dpctl dump-flows
*** s1 ------------------------------------------------------------------------
 cookie=0x0, duration=127.514s, table=0, n_packets=119, n_bytes=11662, priority=100,ip,nw_src=10.0.0.3 actions=drop
mininet>
```

10. GitHub Repository

Include your project repository link here.

Example:

https://github.com/Dhruva-Girish/sdn-dynamic-host-blocking

The repository includes:

- Python controller script
- Project documentation
- Demo screenshots

11. Conclusion

This project demonstrates how Software Defined Networking can be used to dynamically detect and block malicious hosts. By leveraging a centralized controller, network traffic can be monitored and security policies can be enforced in real time.

The implementation successfully detects suspicious activity based on packet behavior and dynamically installs blocking rules in the OpenFlow switch. This approach provides a flexible and programmable method for improving network security.

The project highlights the advantages of SDN in implementing dynamic network security mechanisms and demonstrates the effectiveness of centralized network control.